

MedGuard AI – Project Development Plan

Phase 1 – Requirement Analysis & Planning

🎯 Objectives

- Finalize complete system scope
- Freeze technology stack:
 - Django REST Framework
 - PostgreSQL
 - FAISS
 - Ollama (Local LLM)
 - Docker
 - React (Web)
 - Flutter (Android Mobile)
- Define architecture and modules

📌 Activities

- Study problem statement
- Identify:
 - Functional Requirements
 - Non-Functional Requirements
- Define:
 - Technical constraints
 - Regulatory constraints
 - Security constraints
- Perform risk analysis planning
- Plan dataset collection:
 - Medicine data
 - Legality data (India-focused)
 - Barcode mappings
- Define Web architecture
- Define Mobile architecture:
 - Mobile → API Gateway → Microservices
- Define user roles (User / Admin / Pharmacy future)

✅ Deliverables

- Final SRS Document
- Architecture Blueprint
- Risk Assessment Plan
- Mobile Architecture Flow Diagram

Phase 2 – System Architecture & Database Design

🎯 Objectives

- Design microservice architecture
- Design PostgreSQL schema in 3NF

Microservices Defined

- API Gateway (Django + JWT)
- Medicine Verification Service
- Symptom Remedy LLM Service
- PostgreSQL Database Service
- FAISS Persistent Volume
- Ollama Service

Database Design

Tables:

1. Users
2. Medicines
3. Medicine Barcodes
4. Embeddings Metadata
5. Verification History
6. Symptom History

ENUM Types:

- Legality Status:
 - Legal
 - Banned
 - Restricted
 - Under Review
- Result Status:
 - Authentic
 - Fake
 - Expired
 - Illegal
 - Not Found

Indexing:

- Email
- Barcode
- Medicine Name
- Legality Status
- Timestamp fields

REST Communication Flow

Web → API Gateway → Medicine/Symptom Services

Mobile → API Gateway → Services

Inter-service via Docker network

Deliverables

- ER Diagram

- Logical Database Design
- Service Communication Diagram
- Mobile-Backend Integration Design

Phase 3 – Dataset Preparation & RAG Setup

Objectives

- Prepare medicine dataset
- Implement RAG pipeline

Activities

Dataset Preparation

- Clean medicine dataset
- Normalize names and fields
- Assign legality ENUM values
- Validate barcode mappings

Embedding Generation

- Use Sentence Transformers
- Generate embeddings
- Create text chunks
- Store embedding metadata in PostgreSQL

FAISS Setup

- Build FAISS index
- Store in Docker persistent volume
- Tune similarity threshold
- Optimize for <5 sec search target

RAG Workflow

1. Convert medicine dataset into embeddings
2. Store in FAISS index
3. Convert user input into embedding
4. Perform similarity search
5. Retrieve top-K matches
6. Apply verification logic

Deliverables

- Cleaned Medicine Dataset
- FAISS Vector Index
- Embedding Metadata Records

Phase 4 – Backend Development (Core Microservices)

- ◆ 4.1 API Gateway Development

- User registration
- Login
- JWT authentication
- Token validation
- Role-based access support
- Request routing
- Error handling
- CORS configuration (Web + Mobile ready)
- HTTPS-ready configuration

◆ 4.2 Medicine Verification Service

Functionalities:

1. Multi-modal input:
 - Text
 - Image
 - Barcode
2. OCR integration (Google Vision)
3. Barcode decoding (pyzbar)
4. Regex extraction:
 - MFG
 - EXP
5. RAG similarity search
6. Expiry validation
7. Legality classification
8. Authenticity decision logic
9. LLM-based summary generation
10. Store verification history

Performance Targets:

- Barcode lookup: <2 sec
- FAISS search: <5 sec

◆ 4.3 Symptom Remedy LLM Service

Functionalities:

- Accept symptom input
- Generate:
 - Safe home remedies
 - Proportions
 - Precaution notes
- Add mandatory disclaimer
- Prevent:
 - Diagnosis
 - Prescription drugs
 - Emergency treatment advice

Safety Controls:

- Strict prompt engineering
- Constrained output formatting

✓ Deliverables

- Functional REST APIs
- Authentication system
- RAG integration working
- LLM service integrated

🌐 Phase 5 – Web & Mobile Frontend Development

🌐 5A – Web Frontend (React)

Features:

- Login/Register
- Medicine verification page:
 - Text input
 - Image upload
 - Barcode input
- Display:
 - Authenticity (color-coded)
 - Expiry
 - Legality
 - AI summary
- Symptom remedy page
- Disclaimer display
- Verification history dashboard
- Responsive UI

Deliverable:

- Responsive Web Application

📱 5B – Mobile Application (Flutter – Android)

As defined in SRS Mobile Module

MedGuradAI_SRS

Mobile Functional Capabilities:

1. User registration & login
2. Capture image via device camera
3. Barcode scanning (Google ML Kit)
4. Send captured data to backend
5. Display:

- Authenticity result
- Expiry status
- Legality status
- AI summary
- 6. Symptom input
- 7. Remedy display + disclaimer
- 8. Verification history
- 9. Sync with web platform

Mobile Constraints:

- Android only (initial)
- No FAISS locally
- No LLM locally
- Requires internet
- Camera quality affects OCR
- Network latency affects speed

Deliverables:

- Android APK
- Mobile ↔ Backend integration complete

Phase 6 – Dockerization & Cloud Deployment

Objectives:

- Containerize all services
- Enable scalable production deployment

Activities:

- Create Dockerfiles
- Configure Docker Compose:
 - api-gateway
 - medicine-service
 - symptom-llm-service
 - postgres-db
 - ollama
- Configure persistent FAISS storage
- Configure HTTPS
- Deploy to AWS / DigitalOcean
- Setup:
 - Firewall
 - Environment variables
 - Backup strategy
- Generate signed Android build

Deliverables:

- Dockerized Microservice System

- Live Cloud Deployment
- Persistent Volumes Configured
- Production-ready Android build

● Phase 7 – Testing & Validation

- ◆ Functional Testing
 - Multi-modal validation
 - Expiry accuracy
 - Legality classification
 - RAG similarity correctness
 - LLM safety validation
- ◆ Performance Testing
 - FAISS search under 5 sec
 - 100 concurrent users
 - LLM latency under 10 sec
 - Mobile API latency
- ◆ Security Testing
 - JWT validation
 - SQL injection prevention
 - File upload validation
 - HTTPS enforcement
- ◆ Risk Testing
 - LLM hallucination checks
 - Similarity threshold tuning
 - Fault isolation testing
- ◆ Mobile Testing
 - Android compatibility
 - Camera permission handling
 - Barcode scan accuracy
 - Network interruption testing
 - Web-Mobile data sync validation

Deliverables:

- Test Case Document
- Performance Report
- Bug Fix Logs
- Mobile Test Report

Phase 8 – Optimization & Hardening

Activities:

- Tune FAISS index
- Optimize embedding dimensions
- Improve Docker scaling
- Horizontal scaling setup
- Optimize DB indexing
- Implement caching if required
- Logging & monitoring
- Load balancing

Deliverables:

- Optimized Search Performance
- Stable Production Version

Phase 9 – Documentation & Final Release

Activities:

- API documentation
- Deployment guide
- User manual (Web)
- Mobile App user guide
- Admin manual
- Architecture explanation
- RAG & FAISS explanation
- Viva preparation

Deliverables:

- Final Project Report
- Deployment Manual
- Demo-ready system

Phase 10 – Future Enhancements

- Government API integration
- Pharmacy dashboard
- Counterfeit reporting system
- Multi-language support
- Confidence score visualization
- Advanced AI Health Assistant
- On-device OCR
- iOS Mobile App
- Push notifications
- Biometric login
- Offline caching

FINAL SYSTEM SUMMARY

MedGuard AI is:

- Microservice-based
- Dockerized
- RAG-powered
- FAISS-accelerated
- Local-LLM enabled
- Web + Mobile integrated
- Privacy-preserving
- Scalable healthcare AI platform

It ensures:

- Medicine authenticity validation
- Expiry detection
- Legal compliance awareness
- Safe home remedy assistance
- Secure & scalable architecture